



Pontifícia
Universidade
Católica do
Rio de Janeiro

DEI
DEPARTAMENTO
DE ENGENHARIA
INDUSTRIAL



ENG 4560 – Projeto Integrado VI: Distribuição Física | 2026.1

Aula 3: Modelagem matemática do CVRP (Parte 1)

Prof. Marcello Congro

marcellocongro@puc-rio.br



CTC
CENTRO TÉCNICO CIENTÍFICO / PUC-Rio

Onde estamos no curso?

- Estamos modelando um problema clássico de Logística (CVRP – *Capacitated Vehicle Routing Problem*):



Como planejar rotas de veículos de modo a minimizar o custo total, respeitando restrições operacionais?

- Na aula passada, partimos de uma base de dados operacional, tratamos inconsistências típicas e traduzimos para os elementos que um problema de roteirização exige: nós, demandas, distâncias, custos e tempos:
 - ❖ Isto significa dizer que fizemos uma representação estruturada do sistema logístico;
 - ❖ **Como transformar uma instância bem definida em um modelo matemático que um solver consegue interpretar e resolver?**
 - ❖ Nesta aula, vamos aprender a escrever um modelo de Programação Inteira para o CVRP usando Pyomo, interpretando a representação logística de cada elemento do modelo e como o solver lê a informação para buscar soluções.



<https://prologtransportes.com.br/>

Modelagem matemática

- No contexto de Otimização, modelagem matemática é o **processo de representar decisões reais** (por exemplo, “um veículo visita quais clientes e em qual sequência?”) **por meio de variáveis formais**, e de **traduzir regras operacionais** (por exemplo, “cada cliente deve ser atendido uma vez”, “há limite de capacidade”, etc.) **por meio de restrições matemáticas**.
- Modelos de otimização descrevem o que é permitido acontecer, e não o que vai acontecer. É composto por quatro blocos fundamentais:
 - ❖ **1. Variáveis de decisão**: representam as escolhas;
 - ❖ **2. Função objetivo**: o que é desejável acontecer, mede o custo-benefício da escolha;
 - ❖ **3. Restrições**: o que é permitido acontecer. Caso não sejam definidas, o solver não as conhece;
 - ❖ **4. Domínio de variáveis**: define se elas são contínuas, inteiras ou binárias.
 - ❖ OBS.: Quem “escolhe” dentro do conjunto de possibilidades é o solver. Ele é extremamente eficiente em explorar o espaço de soluções, mas não tem intuição logística.



No CVRP: a decisão binária de usar um arco (i, j) é representada por uma variável 0/1. Cria-se um problema combinatório, pois o n^o de combinações possíveis cresce muito rapidamente com o n^o de clientes!

Programação Linear Inteira (MIP)

- A classe de modelos mais importante para roteirização é a **Programação Linear Inteira Mista** MIP (do inglês, *Mixed-Integer Programming*);
- Nessa classe, tanto a função objetivo quanto as restrições são lineares, mas uma parcela das variáveis é restrita a assumir valores inteiros (muitas vezes binários);
- Em problemas combinatórios, a solução não é encontrada “derivando” uma função, e sim combinando escolhas discretas. Na prática, os *solvers* não enumeram todas as combinações (isso seria inviável), mas constroem uma busca inteligente baseada em relaxações e provas de impossibilidade.

No contexto desta disciplina:

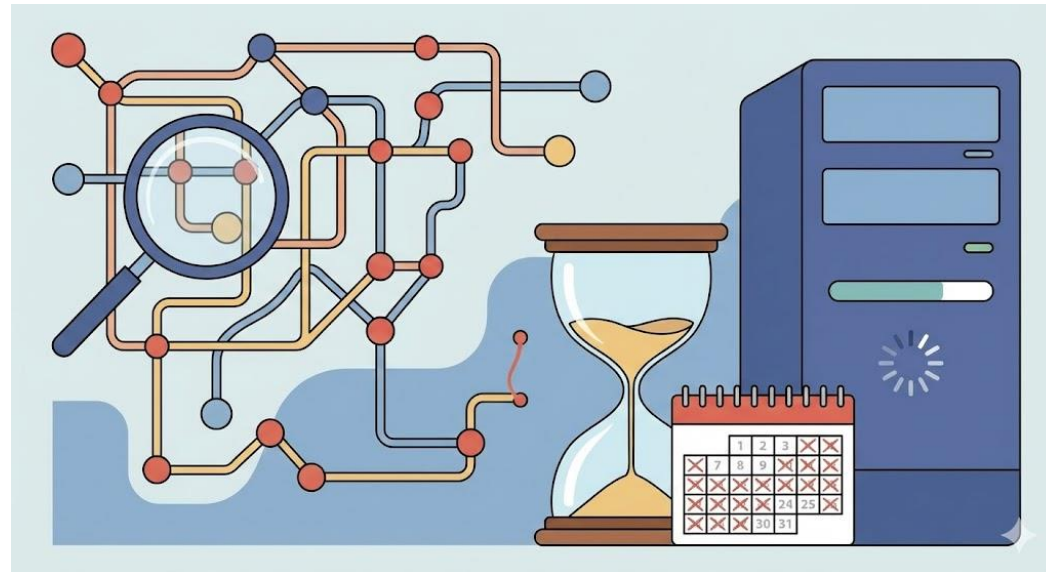
- *MIP é a linguagem operacional que conecta a formulação matemática ao solver;*
- *Pyomo é o ambiente Python que descrevemos o modelo;*
- *O solver é o motor algorítmico que executa a busca (ex.: CPLEX, Gurobi, CBC, etc.)*



Quando dizemos que “o modelo é linear”, isso não significa que o problema é simples: significa que a expressão matemática usa apenas combinações lineares das variáveis, sem produtos entre variáveis e sem funções não lineares!

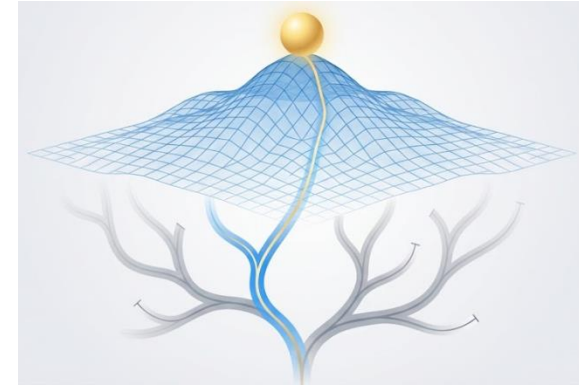
Complexidade e escalabilidade

- O problema de roteirização envolve escolher sequências de visita. Para n clientes, o número de sequências possíveis cresce de forma fatorial;
- No nosso caso, a variável binária associada a arcos (i,j) cresce aproximadamente como $n(n-1)$: para cada par ordenado de nós distintos existe uma possível decisão 0/1 de usar ou não aquele arco;
- Um modelo exato pode demorar horas ou até dias para ser executado, dependendo do solver e do tamanho da instância.



Métodos exatos

- Dizemos que um modelo é exato quando ele é capaz de (i) entregar uma solução viável e (ii) provar que nenhuma outra solução viável tem valor objetivo melhor.
- Essa prova de otimalidade é importante, pois ela permite afirmar que o resultado é globalmente ótimo dentro do modelo formulado.
- A solução não necessariamente é “perfeita no mundo real”. Ela significa que, dadas as variáveis e restrições, não existe alternativa melhor dentro daquele universo matemático.
- Esses métodos são baseados em duas ideias:
 - ❖ **Relaxação:** “tornar o problema mais fácil” temporariamente, geralmente permitindo que variáveis binárias assumam valores fracionários.
 - ❖ **Busca sistemática:** explorar possibilidades de forma organizada, descartando regiões do espaço de solução quando se prova que elas não podem conter soluções melhores que aquelas já encontradas.



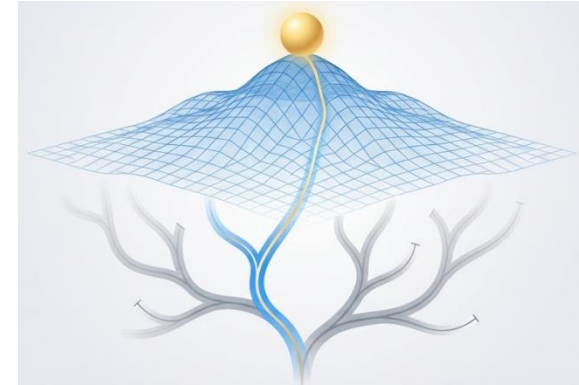
Métodos exatos (cont.)

- Em problemas de otimização como o CVRP, dois conceitos são fundamentais para entender o funcionamento dos solvers MIP:
 - ❖ **Limite inferior (*lower bound*)**: esse valor provém da solução da relaxação linear. Ele representa o melhor valor possível que qualquer solução inteira pode atingir ou melhorar. Nenhuma solução viável inteira poderá ter custo menor que esse limite.
 - ❖ **Limite superior (*upper bound*)**: obtido a partir da melhor solução inteira viável encontrada até o momento pelo algoritmo.
 - ❖ **Gap de otimalidade**: diferença entre o limite superior e inferior, geralmente dado na forma percentual. Quanto menor o gap, mais próxima a solução atual está da solução ótima global.

$$GAP = \frac{UB - LB}{UB}$$



Quando $GAP = 0$, o solver não apenas encontrou uma solução viável, mas provou matematicamente que não existe nenhuma solução melhor dentro do modelo formulado.



- Em problemas de otimização como o CVRP, dois conceitos são fundamentais para entender o funcionamento dos solvers MIP:

❖ **Limite inferior (*lower bound*)**: esse valor provém da solução da relaxação linear. Ele representa o melhor valor possível que qualquer solução inteira pode atingir ou melhorar. Nenhuma solução viável inteira poderá ter custo menor que esse limite.

❖ **Limite superior (*upper bound*)**: obtido a partir da melhor solução inteira viável encontrada até o momento pelo algoritmo.

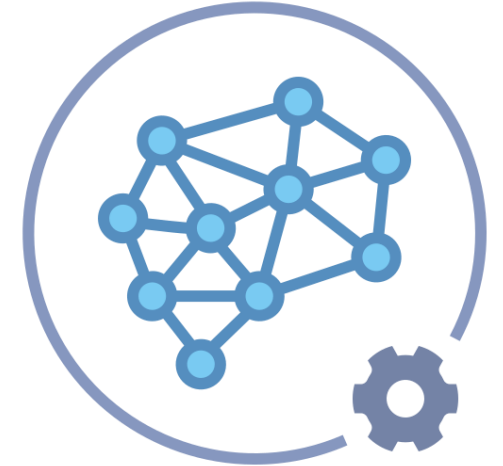
❖ **Gap de otimalidade**: diferença entre o limite superior e inferior, geralmente dado na forma percentual. Quanto menor o gap, mais próxima a solução atual está da solução ótima global.

$$GAP = \frac{UB - LB}{UB}$$

Métodos exatos (cont.)

- *Branch and Bound* (B&B)

- ❖ É o algoritmo base de praticamente todos os solvers MIP modernos;
- ❖ O solver alterna entre resolver problemas “mais fáceis” (relaxações lineares) e impor decisões discretas de forma progressiva, construindo uma árvore de busca.

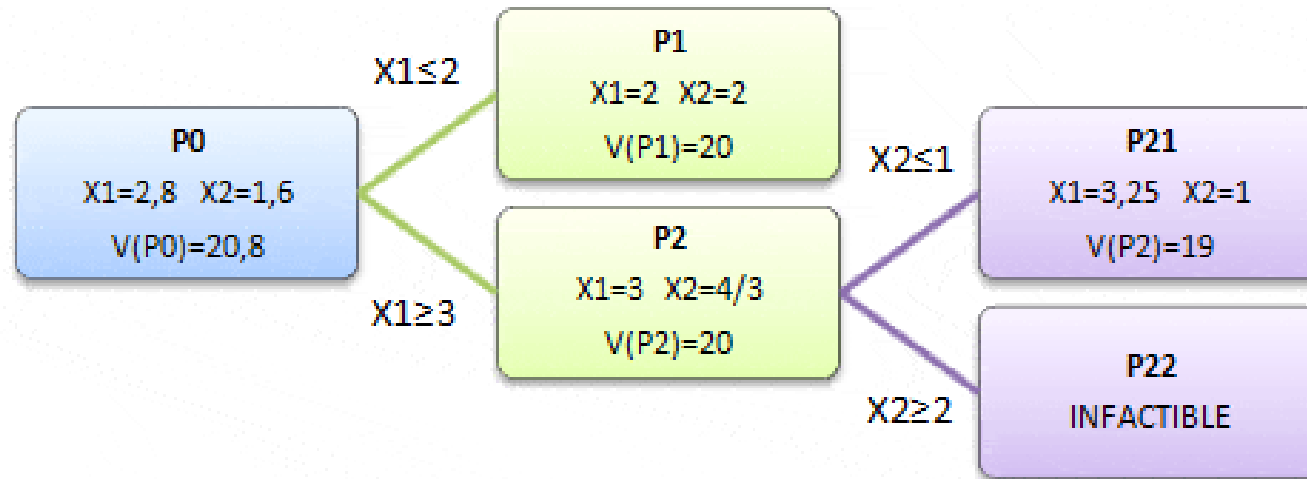


- Como o algoritmo funciona?

- ❖ 1. Resolve o modelo ignorando a restrição de números inteiros (ex.: solução fracionária, $x_{ij} = 0,3$).
- ❖ 2. Escolhe uma variável fracionária e divide o problema em dois subproblemas (nós). Ex.: Força $x_{ij} = 0$ de um lado e $x_{ij} = 1$ do outro.
- ❖ 3. O solver repete o processo (resolve, avalia, ramifica), criando uma “árvore de busca”;
- ❖ 4. Ao encontrar uma solução inteira válida, guarda esse valor (a incumbente). Em minimização, esse será o nosso limite superior;
- ❖ 5. Se a relaxação de um nó gerar um resultado pior (ou igual) à melhor solução já guardada, o solver “corta” esse galho da árvore, pois provou que não há melhor solução ali.

Métodos exatos (cont.)

- Branch and Bound (B&B)



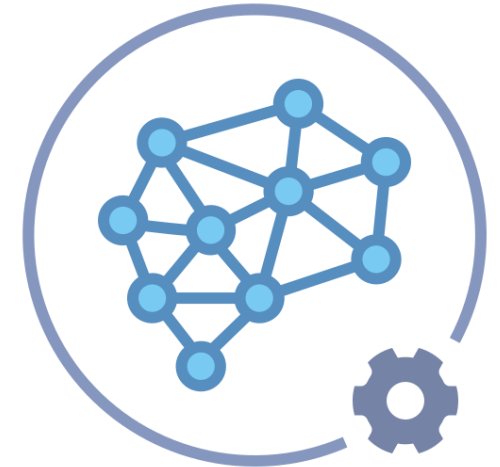
Métodos exatos (cont.)

- *Branch and Cut* (B&C)

- ❖ É o algoritmo de evolução natural do B&B, usado em solvers comerciais (Gurobi, CPLEX);
- ❖ Além de ramificar, o solver também tenta fortalecer a relaxação linear adicionando restrições válidas chamadas de cutting planes (cortes).

- Como o algoritmo funciona?

- ❖ 1. Resolve o modelo ignorando a restrição de números inteiros
- ❖ 2. Se a solução for fracionária, o solver procura por planos de corte. São novas restrições matemáticas que eliminam a solução fracionária atual, mas não cortam nenhuma solução inteira válida.
- ❖ 3. O modelo é resolvido novamente com esse novo corte. Esse ciclo (resolver e cortar) se repete até que não seja mais possível (ou eficiente) encontrar novos cortes.
- ❖ 4. Se os cortes se esgotaram e a solução ainda for fracionária, o solver divide o problema em dois subproblemas (cria nós na árvore).
- ❖ 5. Se a relaxação de um nó gerar um resultado pior (ou igual) à melhor solução já guardada, o solver “corta” esse galho da árvore, pois provou que não há melhor solução ali.



Considerando el conjunto $\{1, 2, 3, 4, 5\}$
se busca el subconjunto que sume 10:

Solvers na prática

- Embora o núcleo algorítmico (B&B/B&C) seja conceitualmente comum, os *solvers* diferem muito em desempenho, robustez e recursos:
 - ❖ **CBC** é um solver open-source amplamente utilizado em ambientes acadêmicos por ser gratuito e relativamente fácil de instalar;
 - ❖ Solvers comerciais como **Gurobi** e **CPLEX** investem pesado em estratégias de busca, seleção de variáveis para ramificação, geração de cortes, paralelização e pré-processamento. Na prática, isso frequentemente resulta em diferenças de tempo de solução muito grandes, especialmente em instâncias mais difíceis.
 - ❖ No contexto do *Google Colab*, a escolha do **CBC** é pragmática. Ele pode ser instalado via apt-get, não exige licenças e funciona bem integrado a ambientes Python com Pyomo.



GUROBI
OPTIMIZATION



Solvers na prática (cont.)

- Ao executar um solver, o resultado mais importante não é o resultado da função objetivo, e sim a **interpretação completa** do relatório de solução. Você deverá sempre avaliar:
 - ❖ O **status do solver**, que indica se o processo de solução foi executado corretamente;
 - ❖ A **condição de término**, que indica se o solver encontrou ótimo, viabilidade sem prova de ótimo, inviabilidade, limite de tempo ou algum outro tipo de encerramento;
 - ❖ O **tempo de processamento**, pois ele revela o custo computacional da modelagem para aquela instância.
- A escolha do solver influencia diretamente desempenho e robustez: um solver *open-source* pode resolver instâncias pequenas com eficiência razoável, mas pode demandar maior tempo para instâncias médias e grandes; solvers comerciais tendem a ser superiores em estratégias de busca, cortes, pré-processamento e paralelização.



GUROBI
OPTIMIZATION



Licença acadêmica PUC-Rio

- GUROBI (precisa ter e-mail com domínio @aluno.puc-rio.br)
 - ❖ <https://www.gurobi.com/academia/academic-program-and-licenses/>
- CPLEX (licença gratuita por 30 dias com limitação de 1000 variáveis e 1000 restrições)
 - ❖ <https://www.ibm.com/account/reg/br-pt/signup?formid=urx-20028>



GUROBI
OPTIMIZATION



Definição formal do problema CVRP

- O *Capacitated Vehicle Routing Problem* (CVRP) pode ser formalmente definido como um problema de otimização combinatória em que se deseja determinar um conjunto de rotas que partem e retornam a um depósito central, atendendo um conjunto de clientes com demandas conhecidas, de modo a minimizar o custo total de operação, respeitando restrições de capacidade dos veículos.
- Seja:
 - ❖ $N = \{0, 1, \dots, n\} \Rightarrow$ conjunto de nós, onde 0 representa o depósito;
 - ❖ $C = \{1, 2, \dots, n\} \Rightarrow$ conjunto de clientes definidos na instância;
 - ❖ $q_i \Rightarrow$ demanda do cliente i ;
 - ❖ $Q \Rightarrow$ capacidade do veículo;
 - ❖ $c_{ij} \Rightarrow$ custo de deslocamento do nó i para o nó j .
- O objetivo é determinar quais arcos (i, j) devem ser utilizados de modo que cada cliente seja visitado exatamente uma vez, respeitando as restrições operacionais e minimizando o custo total.



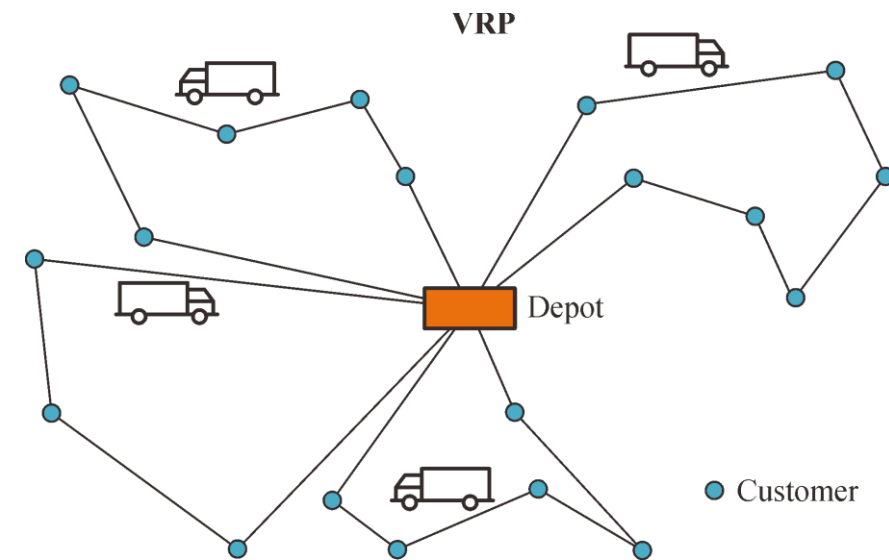
Variáveis de decisão

- A principal variável de decisão do modelo é associada ao uso de arcos.

$$x_{ij} = \begin{cases} 1, & \text{se o arco } (i,j) \text{ é utilizado} \\ 0, & \text{caso contrário.} \end{cases}$$

- Essa variável é binária e representa uma decisão logística concreta: se um veículo sai do nó i e vai diretamente ao nó j .
- Com isso, uma rota completa pode ser vista como uma coleção de arcos com $x_{ij} = 1$.
- É preciso decidir quantos veículos serão utilizados, e contaremos quantas rotas saem do depósito.

$$m = \sum_{j \in C} x_{0j}$$



O valor “ m ” representa o nº de veículos acionados, pois cada veículo que opera precisa necessariamente sair do depósito em direção ao primeiro cliente de sua rota.

Função objetivo

- O objetivo é minimizar o custo total, composto por dois termos principais: o custo de deslocamento (proporcional à distância) e o custo fixo diário por veículo acionado.
- O custo variável é obtido somando o custo c_{ij} de cada arco utilizado. Como $x_{ij} = 1$, indica que o arco foi escolhido, a soma a seguir representa o custo total de deslocamento:

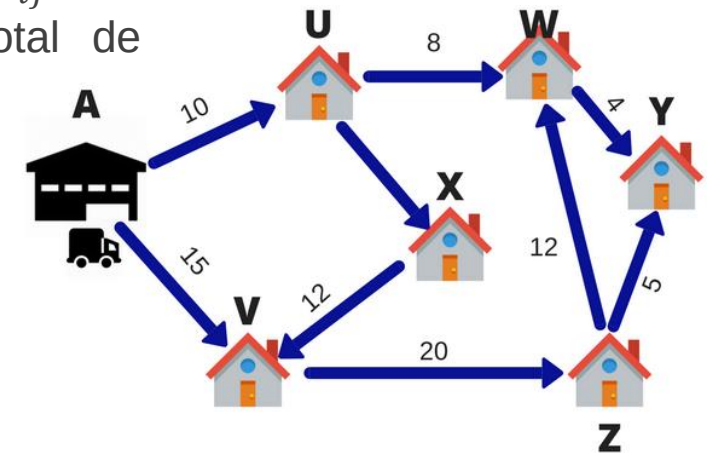
$$\sum_{i \in N} \sum_{j \in N, j \neq i} c_{ij} x_{ij}$$

- O custo fixo depende do número de veículos “m”. O custo fixo total será:

$$f * m = f \sum_{j \in C} x_{0j}$$

- Função objetivo completa:

$$\min \left[\sum_{i \in N} \sum_{j \in N, j \neq i} c_{ij} x_{ij} + f \sum_{j \in C} x_{0j} \right]$$



- Se o custo fixo “f” for alto, o solver tende a usar menos veículos e aceitar rotas maiores (desde que viáveis).
- Se “f” for baixo, o solver pode preferir abrir mais veículos para reduzir custo variável (rotas mais “compactas”)

Hipóteses simplificadoras (aula de hoje)

- Na aula de hoje, vamos adotar uma **formulação introdutória** do CVRP, com foco em entender a estrutura do modelo e o papel de cada bloco (variáveis, função objetivo, restrições e domínio). Deste modo:
 - ❖ **Frota homogênea (apenas VUC):** Assumimos que todos os veículos utilizados têm a mesma capacidade e o mesmo custo fixo. $Q = 3000$ kg (capacidade) e $f = \text{R\$ } 550,00$ (custo fixo diário);
 - ❖ **Sem identificação individual de veículos** (modelo ainda não “rotula” veículos): O modelo decide quais arcos (i,j) são usados, mas ainda não representa explicitamente “veículo k faz a rota k ”. Por isso, várias restrições operacionais serão tratadas de forma global (agregada), e não por rota;
 - ❖ **Jornada (tempo) como restrição agregada:** quando usamos tempo, ele é aplicado de forma agregada, como uma checagem global do “orçamento de horas” da operação, e não como jornada por veículo/rota.



Hipóteses simplificadoras (cont.)

- Com as restrições apresentadas até este ponto, o modelo assegura que: cada cliente é visitado exatamente uma vez; o número de entradas e saídas em cada cliente é consistente; o depósito equilibra partidas e retornos, e portanto o número de veículos acionados pode ser inferido pelo número de saídas do depósito.
- A capacidade é tratada de forma global (agregada);
- O objetivo é garantir apenas que a capacidade total disponível na operação seja suficiente para atender à demanda total dos clientes;
- A restrição utilizada compara a soma das demandas de todos os clientes com a capacidade total disponível (capacidade do veículo x número de veículos utilizados);
- O número de veículos é estimado implicitamente a partir do número de saídas do depósito;
- Essa abordagem garante viabilidade global da operação, mas não assegura que cada rota individual respeite a capacidade real do veículo.



Podem surgir rotas que, do ponto de vista operacional, transportariam mais carga do que seria possível em um único VUC.

Hipóteses simplificadoras (cont.)

- Jornada diária operacional (referência do problema real): adotamos $H = 8$ horas como limite típico de jornada de um veículo na operação.
- Se “m” é o número de veículos acionados, então o total de horas disponíveis na operação é aproximadamente $H \cdot m$.
- Esse limite **não é imposto no modelo matemático** (MIP). O modelo será resolvido sem restrição de tempo/jornada.
- Em vez de impor a jornada como restrição, nós a utilizaremos como um critério de validação pós-solução: após obter a solução ótima do MIP, calcularemos o tempo total de cada rota reconstruída (tempo de deslocamento + tempo de serviço) e verificaremos se alguma rota excede 8 horas.



Restrições

Atenção! Essas restrições garantem o atendimento individual, mas não garantem, sozinhas, que todos os clientes estejam conectados ao depósito por rotas válidas!

- Restrição de visita única

- ❖ Cada cliente deve ser atendido exatamente uma vez (uma função controla as saídas e outra as entradas);
- ❖ A **restrição de saída** impõe que, para cada cliente $i \in C$, deve existir exatamente um arco saindo de i com $x_{ij} = 1$. Em linguagem matemática:

$$\sum_{j \in N, j \neq i} x_{ij} = 1 \quad \forall i \in C$$

- ❖ Após o veículo atender ao cliente i , ele deve seguir para um próximo nó (seja ele outro cliente ou o depósito).
- ❖ A **restrição de entrada** impõe que, para cada cliente $j \in C$, deve existir exatamente um arco entrando em j . Em linguagem matemática:

$$\sum_{i \in N, i \neq j} x_{ij} = 1 \quad \forall j \in C$$

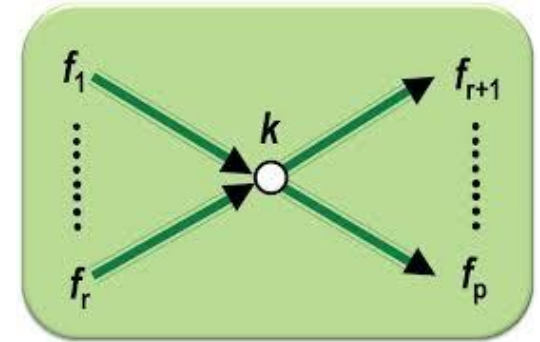
Restrições (cont.)

- Restrição de conservação de fluxo

- ❖ Para cada nó intermediário, o fluxo que entra deve ser igual ao fluxo que sai;
- ❖ Isso significa que, se um veículo chega a um cliente, ele necessariamente deve sair dele. Em termos matemáticos, para cada cliente $i \in C$:

$$\sum_{j \in N, j \neq i} x_{ij} - \sum_{j \in N, j \neq i} x_{ji} = 0$$

- ❖ O cliente não pode ser “ponto final” de uma rota incompleta (exceto no caso do depósito). A conservação do fluxo garante ciclos direcionados/caminhos fechados.



Atenção! A conservação de fluxo ainda não assegura que todos os clientes estejam conectados ao depósito!

Restrições (cont.)



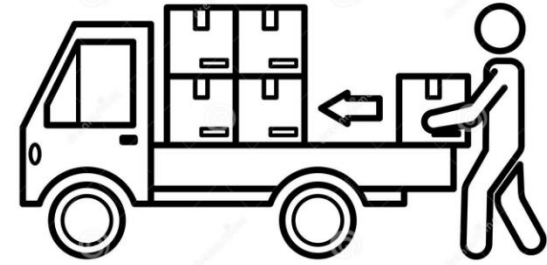
- Restrição no depósito/CD

- ❖ O depósito é o ponto de origem e destino de todas as rotas. O equilíbrio imposto para os clientes precisa ser adaptado para o nó 0.
- ❖ O número de veículos que saem do depósito deve ser igual ao número de veículos que retornam. Formalmente, escrevemos:

$$\sum_{j \in C} x_{0j} = \sum_{i \in C} x_{i0}$$

- ❖ O lado esquerdo dessa equação representa o número total de veículos utilizados, enquanto o lado direito representa o número total de retornos ao centro de distribuição.

Restrições (cont.)



- Capacidade agregada

- ❖ Cada veículo possui um limite máximo de carga Q , e a soma das demandas atendidas em uma rota não pode ultrapassar esse valor;
- ❖ A capacidade total disponível deve ser suficiente para atender a demanda total exigida. Se q_i é a demanda do cliente i , a demanda total pode ser escrita como:

$$\sum_{i \in C} q_i$$

- ❖ Se utilizamos “ m ” veículos, cada um com capacidade Q , a capacidade total é $Q \cdot m$. Impomos então que:

$$\sum_{i \in C} q_i \leq Q \cdot m$$

- ❖ O número de veículos acionados deve ser suficiente para transportar toda a carga. Se a demanda total aumentar, então o modelo vai precisar aumentar “ m ” para que a desigualdade seja satisfeita.

Restrições (cont.)

- Exclusão de autoarcos

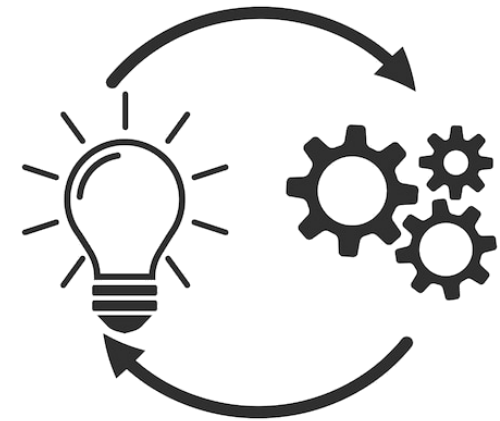
- ❖ Não faz sentido que um veículo saia de um nó e retorne imediatamente ao mesmo nó sem deslocamento. Logo, precisamos impor que:

$$x_{ii} = 0 \quad \forall i \in N$$

- ❖ Embora essa restrição possa parecer trivial, ela evita soluções degeneradas e simplifica a interpretação do grafo resultante;
- ❖ Em modelos de roteirização, pequenos detalhes estruturais podem impactar significativamente o comportamento do solver.

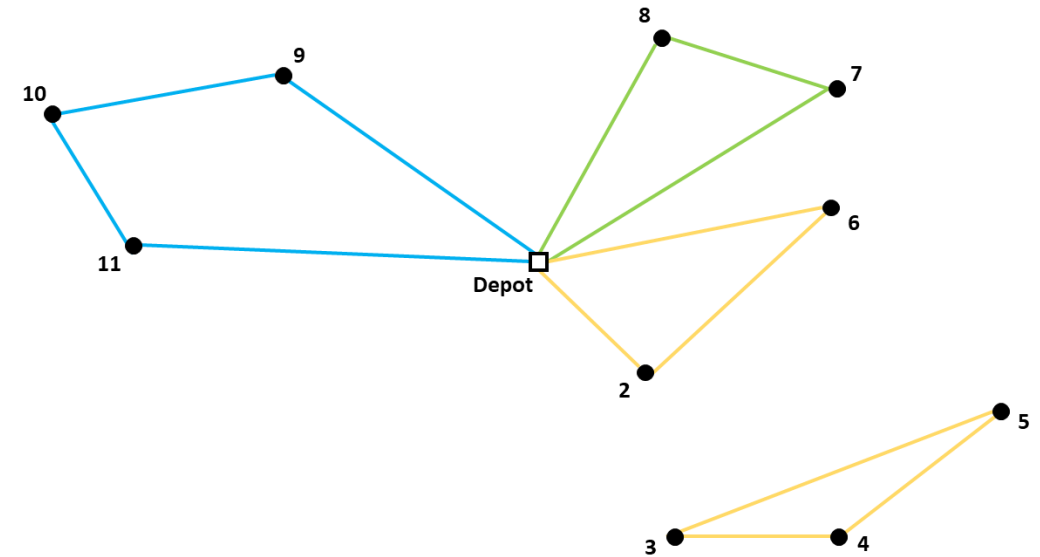
O que o modelo já garante até aqui

- Com as restrições apresentadas até este ponto, o modelo assegura que:
 - ❖ Cada cliente é visitado exatamente uma vez;
 - ❖ O número de entradas e saídas é consistente;
 - ❖ O depósito equilibra partidas e retornos;
 - ❖ A capacidade total é suficiente;
 - ❖ O tempo total será avaliado após a reconstrução das rotas.
- Se analisarmos apenas essas propriedades, poderíamos imaginar que o problema está completamente modelado. Entretanto, existe um fenômeno estrutural importante que ainda não foi eliminado: a possibilidade de **formação de ciclos desconectados do depósito**.



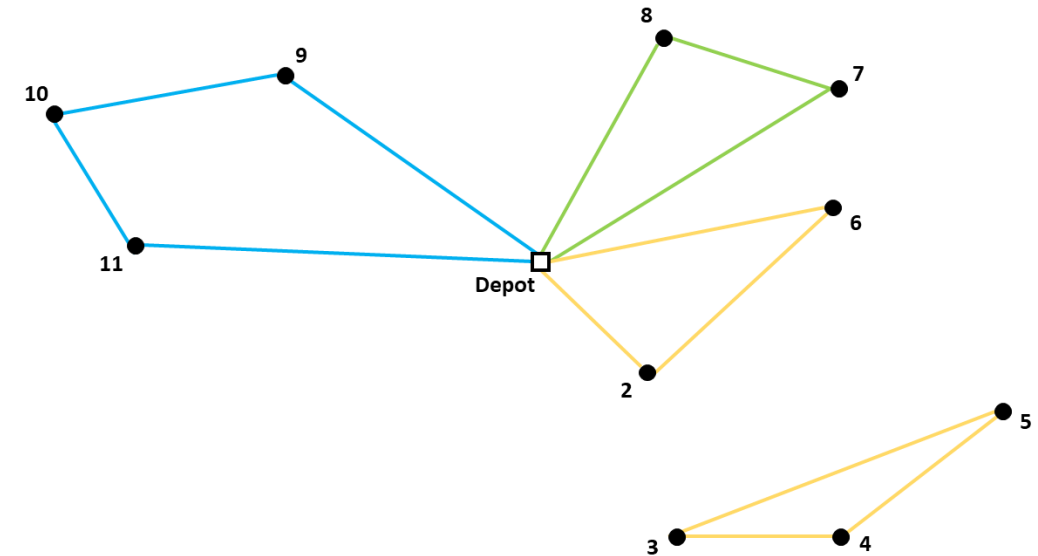
Subtours

- Um *subtour* é um ciclo fechado que envolve apenas clientes e que se não conecta ao depósito
 - ❖ Todas as restrições de entrada, saída e conservação de fluxo são satisfeitas, mas nenhum dos nós envolvidos está conectado ao depósito;
 - ❖ Matematicamente, o modelo permite esse ciclo porque as restrições impostas até agora garantem apenas propriedades locais;
 - ❖ Um veículo não pode “aparecer” em um subconjunto de clientes sem ter saído do depósito.
- O surgimento de subtours é um exemplo clássico da diferença entre coerência estrutural local e conectividade global.



Subtours (cont.)

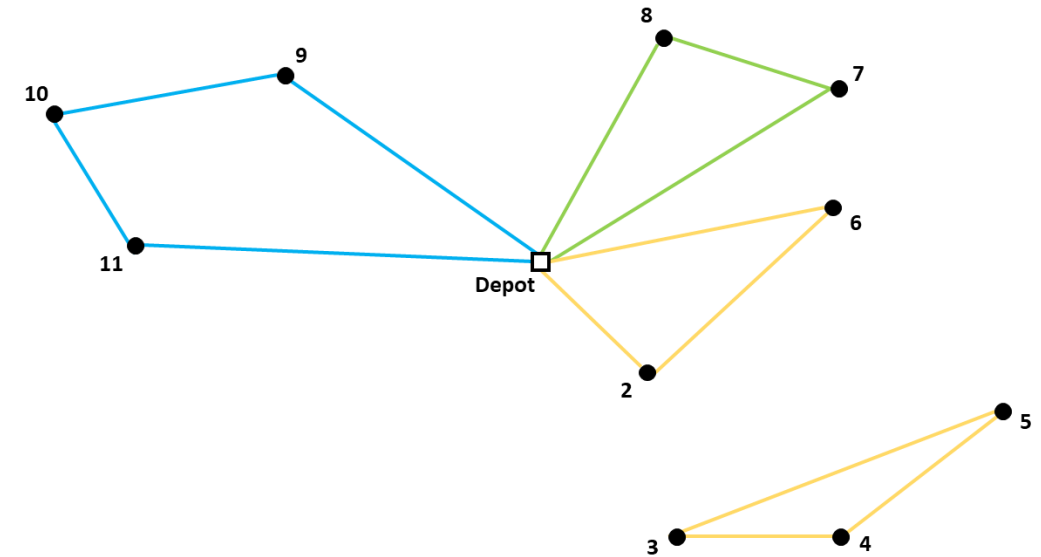
- Eliminar *subtours* significa garantir que qualquer subconjunto de clientes esteja conectado ao depósito por pelo menos um arco que o ligue ao restante da rede
 - ❖ Formulação mais conhecida é o **MTZ** (Miller-Tucker-Zemlin), que introduz uma variável contínua u_i associada a cada cliente, para impor uma ordem ou nível implícito nos clientes visitados, impedindo a formação de ciclos internos.
 - ❖ A ideia é que, se um arco $i \rightarrow j$ for utilizado, então o nível de j deve ser maior que o nível de i .
 - ❖ Cada estratégia (existem outras além do MTZ) possui impacto direto no tamanho do modelo e no tempo de solução.
- Adota-se um número polinomial de restrições e variáveis (sim



$$u_i - u_j + nx_{ij} \leq n - 1$$

Subtours (cont.)

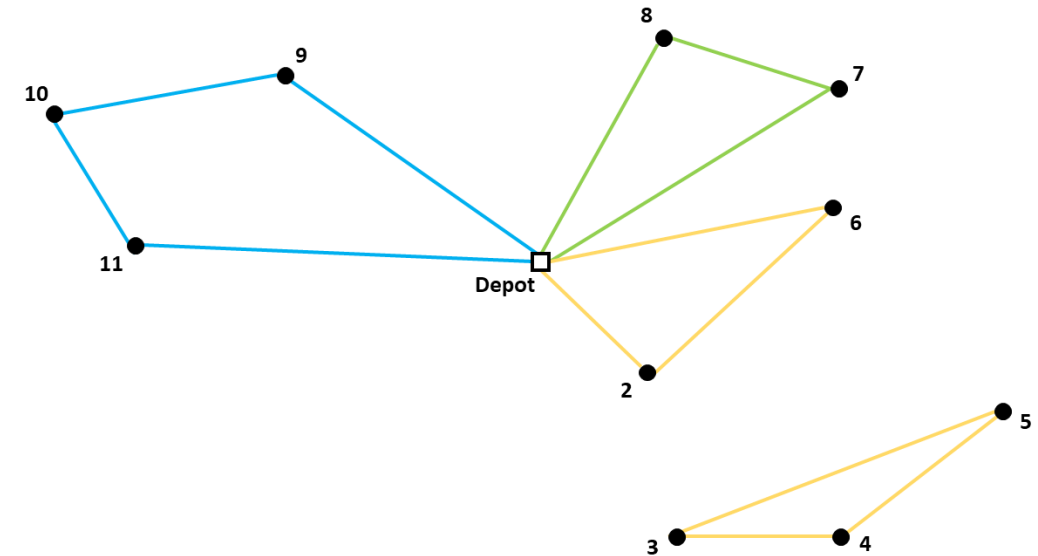
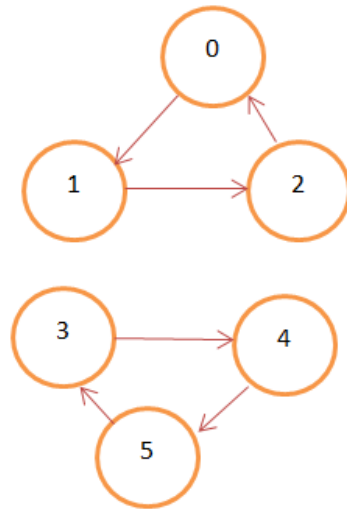
- Logo após a solução, extraímos os arcos (i, j) para os quais a variável binária foi ativada, ou seja, aqueles que o solver escolheu como parte da solução;
- Em seguida, construímos uma relação de sucessão (um “próximo nó” para cada nó), o que permite percorrer o grafo induzido pela solução;
- Depois, tentamos reconstruir rotas iniciando pelas saídas do depósito, isto nos dá as “rotas principais” que de fato passam pelo depósito;
- Por fim, fazemos uma varredura para identificar ciclos que não incluem o depósito
 - Se existir um conjunto de clientes que forma um ciclo fechado entre si e que não é alcançado pelo depósito, temos um subtour



*Subtour não é um “bug” do solver!
“Resolver” não significa resolver corretamente o problema real, mas sim o problema matemático que foi escrito.*

Subtours (cont.)

- Uma vez resolvido o modelo, podemos analisar os arcos selecionados e reconstruir os ciclos formados;
- Nesta aula, o subtour é tratado como diagnóstico e evidência de limitação estrutural (que deverá ser corrigida).



Checagem por rota reconstruída

- Após resolver o MIP, o notebook extrai os arcos selecionados (i, j) para os quais $x[i, j] = 1$. Esses arcos formam um grafo dirigido que representa a solução matemática encontrada.
- Em seguida, o código deve reconstruir rotas iniciando pelas saídas do depósito. Cada saída do nó 0 é interpretada como o início de uma rota candidata. A rota é obtida seguindo os sucessores até retornar ao depósito.
- Para cada rota reconstruída, calculamos:
 - **tempo de deslocamento**: soma dos tempos $T[i, j]$ ao longo dos arcos da rota;
 - **tempo de serviço**: soma dos tempos de atendimento $s[i]$ para os clientes visitados na rota;
 - **tempo total**: deslocamento + serviço.
- Por fim, comparamos o tempo total da rota com $H = 8h$. Se alguma rota exceder esse limite, registramos a violação como evidência de que o modelo, nesta etapa, ainda não impõe viabilidade temporal por rota/veículo.

Hands-on

- **2ª parte:** Implementação no ambiente Colab



Desafio

- Como você modificaria o modelo para considerar dois tipos de veículos?
- Quais alterações seriam necessárias nas variáveis, na função objetivo e nas restrições?
 - ❖ **VUC:** $Q = 3000$; $f = 550$
 - ❖ **Fiorino:** $Q = 650$; $f = 250$

